CMPT 450 · SIMON FRASER UNIVERSITY

# Beyond the GPU

## A Microarchitectural Analysis of LLM Tool-Calling Kernels

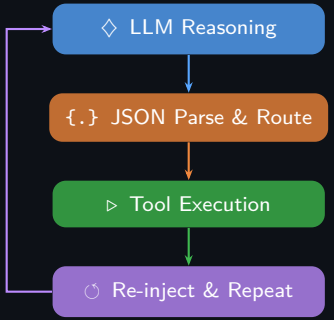
Kunpeng Zhang | Fazal Rehman | TJ Grewal

April 2026

# The Agentic AI Revolution

LLMs have evolved from passive text generators to **autonomous agents** that:

- Invoke external APIs and tools
- Execute multi-step **ReAct** loops
- Parse structured JSON outputs
- Maintain growing context windows




## The Problem:

While the GPU has been relentlessly optimized, the **CPU orchestration overhead** between reasoning steps remains largely *uncharacterized*.

# Research Question

? CORE QUESTION

**How much latency do agentic frameworks silently impose, and can classical hardware optimizations overcome it?**

◇ METHODOLOGY

1. Decoupled profiling harness isolating CPU from GPU
2. PAPI hardware telemetry: cycles, IPC, L3 misses, branch mispredictions

□ KEY FINDINGS

3. Quantify the "Agentic Tax" across compute, memory, I/O domains
4. Identify the "Masking Effect" — kernel wins vanish at system level

# System Architecture

Three strictly isolated layers:

## 1. Orchestrator (Python/FastAPI)

Prompt assembly, tokenization, JSON routing

## 2. Execution Kernels (C++)

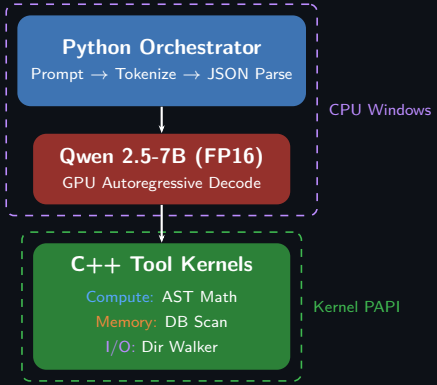
Native binaries with internal PAPI profiling

## 3. Static Workloads

Pre-generated payloads for deterministic runs

## Hardware Telemetry via PAPI:

- Total Cycles & Instructions → IPC
- Branch Mispredictions
- L3 Cache Misses



# Tool Kernels

## Compute-Bound

AST Math Evaluator

Deeply nested expressions stress the **branch predictor** and integer ALUs

→ Control-flow divergence

## Memory-Bound

SQLite Full-Table Scan

Bypasses indexing to intentionally **thrash the LLC** and saturate DRAM

→ Cache miss penalty

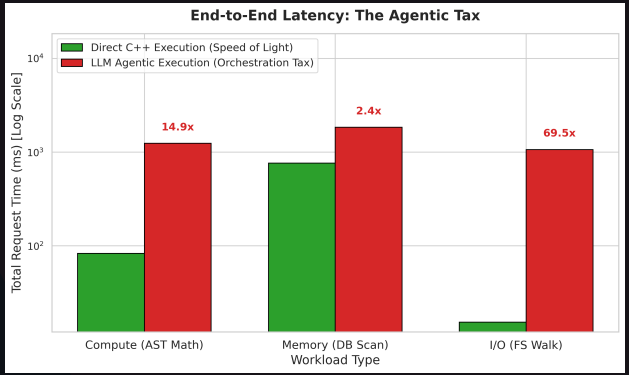
## I/O-Bound

Recursive Dir Walker

POSIX filesystem traversal stresses **OS context switching** and storage

→ Syscall overhead

# Result 1: The Agentic Tax




**69.5×**
I/O Slowdown

**14.9×**
Compute Slowdown

**2.4×**
Memory Slowdown

# Interpreting the Agentic Tax

**▲ KEY INSIGHT**

The Agentic Tax is **not a proportional scalar** — it is a **massive fixed-cost penalty** that disproportionately punishes fast, lightweight tools.

## I/O Kernel (69.5×)

Native execution: ∼15ms

GPU generation + Python serialization dwarf the actual filesystem walk

## Compute Kernel (14.9×)

Fixed orchestration cost still dominates over the AST math evaluation time

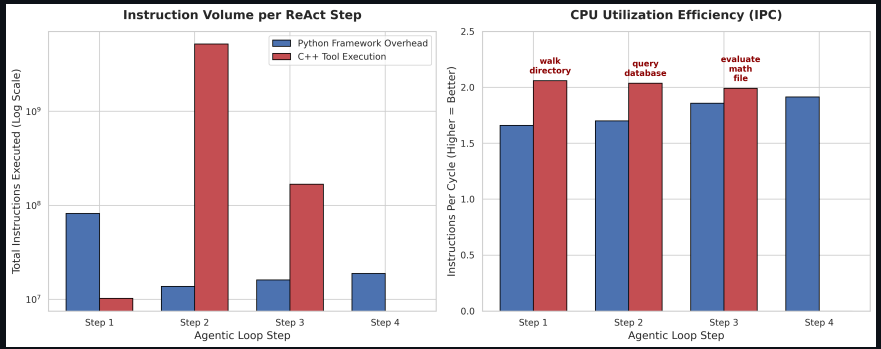
## Memory Kernel (2.4×)

Full-table scan requires >2.5 billion cycles (∼764ms natively), so orchestration is a smaller fraction

## Amdahl's Law at Work

The heavier the native workload, the less proportional impact the fixed overhead has

# Result 2: Multi-Step Workflow Analysis




Left: Instruction volume (log scale). Right: IPC efficiency gap between framework and native tools.

# Multi-Step Insights

## Framework Overhead

⋆ Cold Start: 82M instructions

One-time JSON schema parsing, memory allocation, async env setup

↗ Linear Growth

13.7M → 18.8M instructions as context window re-serializes on each step

↑ IPC Improves: 1.65 → 1.91

String concatenation keeps pipeline fuller than initial dictionary lookups

## C++ Tool Execution

★ Near-Theoretical Max

Compiled kernels operate at ∼2.0 IPC

⊞ DB Scan: 2.03 IPC

Hardware L3 prefetcher streams sequential data effectively

⊠ AST Eval: 1.99 IPC

Slight dip from branch mispredictions in nested if/else parsing

# Optimization 1: Vectorization Wall

## Hypothesis

AVX2 (256-bit SIMD) should accelerate numerical parsing throughput

## Reality

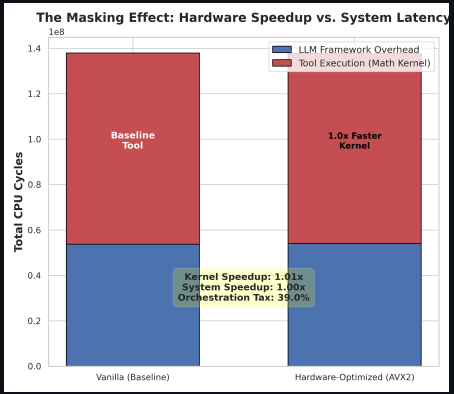
Recursive control-flow divergence and pointer-chasing **completely neutralize** auto-vectorization

× Result:

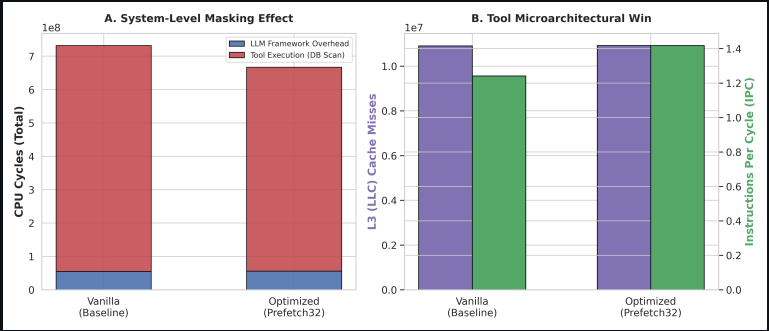
IPC stagnant at ∼1.99

Only 0.6% cycle reduction

String-heavy agentic tools are **intrinsically scalar**



# Optimization 2: Software Prefetching

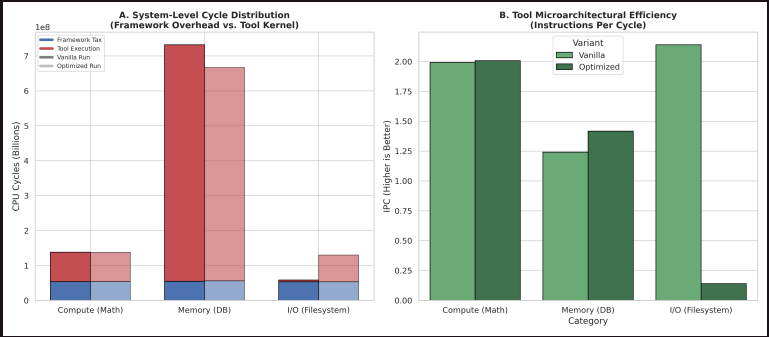



**+14.2%**
IPC Improvement

**66M**
Cycles Saved

**~50ms**
Wall-Clock Saving

# Result 3: The Masking Effect




**Despite a 14.2% kernel IPC gain and 50ms saved,**

**end-to-end system speedup remains $<1\%$**

Total request latency: $\sim$6.3s — orchestration bloat swallows all kernel victories

# Future Work

### ∗ Speculative Execution

Dispatch tools *before* the LLM confirms intent. Overlap CPU tool latency with GPU prefill (PASTE framework). "Act while thinking."

### ⊞ Semantic Caching

Cache tool results with TTL-aware eviction (VAAC algorithm). Over 40% of tool calls are redundant in production. Up to 34% latency reduction.

### ⇄ JIT Model Routing

Dynamic stage-wise routing via Aragog. Different workflow stages need different model capabilities. Up to 217% throughput increase.

### </> Binary Protocols

Replace JSON with SIMD-accelerated parsers (simdjson) or native binary formats (Apache Arrow). Eliminate the CPU serialization bottleneck entirely.

# Conclusion

## The orchestration layer is the bottleneck.

- ✓ Up to **69.5**× latency penalty for lightweight tools
- ✓ Framework overhead **compounds linearly** with multi-step autonomy
- ✓ Kernel optimizations yield <**1% system improvement** — the Masking Effect
- ✓ Solution requires **orchestrator-engine co-design**, not faster tools

*"Until the orchestration layer is treated as a high-performance networking data plane, the potential of hardware-accelerated agentic tools will remain hidden beneath the execution tax."*

# Thank You

Questions?

Kunpeng Zhang · Fazal Rehman · TJ Grewal

Simon Fraser University · CMPT 450